

My annotation is basically that this is very impressive: ChatGPT was able to provide me with conceptual help, helpful papers, and practice questions.

Here is my assignment: (E. Can do at most twice) Make AI-enhanced learning tools for individual concepts and lectures. There are now a plethora of new tools out there (e.g. Guided Learning in Gemini, notebookLM by Google, Study Mode in ChatGPT, Learning mode in Claude, etc... Go ahead and explore/use what's out there) and there are also ways that one can prompt and engage with LLMs even without any special modes that can help you learn on your own. For this option, you should be creating (and sharing with your classmates using an Ed post) whatever prompt/artifact/etc. so that they can also engage in this self-learning together with an interaction trace of you with the tool that shows how it can potentially help. This trace should be annotated critically by you pointing out where it is hallucinating, wrong, or misleading --- as well as places where you think it is doing a good job. One way to think about what you're trying to achieve is as an active AI replacement/substitute for the traditional approach of pre-lecture (or post-lecture) readings that you do to understand the material better before you delve into the homework or your own projects., What I want you (ChatGPT) to do is help me understand Adam

ChatGPT said:

Cool, let's make Adam *really* click. I'll:

1. Build intuition for what Adam is doing.
2. Write the exact update equations and explain every term.
3. Compare to SGD, Momentum, and RMSProp so it's not just "more symbols."
4. Give you a reusable "*AI tutor prompt*" you can use for your assignment to turn this into a self-learning tool.

1. Why do we even need Adam?

Start with **vanilla gradient descent**:

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} L(\theta_t) \quad \theta_{t+1} = \theta_t - \alpha \nabla_{\theta} L(\theta_t)$$

- θ : parameters (weights)
- α : learning rate

- $\nabla_{\theta} L$: gradient of loss

Problems:

- Same learning rate for all parameters → some directions want big steps, some want tiny.
- No memory of past gradients → if gradients keep pointing roughly the same way, we'd like to "accelerate"; if they oscillate, we'd like to slow down.

People tried to fix these:

1. **Momentum**: "remember" past gradients (smoother, faster direction).
2. **RMSProp**: scale steps per-parameter by how big their gradients have been (adaptive learning rate).

Adam = Momentum + RMSProp + some extra fixes.

Think of Adam as:

"For each parameter, keep track of (1) an average gradient and (2) an average squared gradient, correct their early bias, and use them to choose a smart step."

2. Adam: the actual equations

At each time step t (each update):

- Parameters: θ_t
- Gradient: $g_t = \nabla_{\theta} L(\theta_t)$
- Hyperparameters:
 - Learning rate: α (often 10^{-3})
 - $\beta_1 \approx 0.9$ → how much momentum
 - $\beta_2 \approx 0.999$ → how much we smooth squared gradients

- $\epsilon \approx 10^{-8}$ → tiny number to avoid division by zero

We maintain two **state vectors** (same shape as θ):

- m_t : first moment estimate (mean of gradients)
- v_t : second moment estimate (mean of squared gradients)

Step 1: Exponential moving averages of gradient and squared gradient

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Where:

- g_t^2 is elementwise square.
- If $\beta_1 = 0.9$: “90% old + 10% new” gradient.
- If $\beta_2 = 0.999$: “99.9% old + 0.1% new” squared gradient.

Intuition:

- $m_t \approx$ momentum: smoothed gradient.
- $v_t \approx$ smoothed “size” of the gradient; big if gradient is usually large in that coordinate.

Step 2: Bias correction

Because we start with $m_0=0, v_0=0$, early on these are biased toward zero. So Adam does:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Why?

If you repeatedly apply an EMA starting at 0, the expected value is “too small” at early steps because you initialized at 0. Dividing by $1 - \beta^t$ corrects that.

You can think of:

- $m^t_{\hat{m}}_t$: “unbiased” average gradient.
- $v^t_{\hat{v}}_t$: “unbiased” average squared gradient.

Step 3: Parameter update

$$\theta_{t+1} = \theta_t - \alpha \cdot m^t_{\hat{m}}_t + \epsilon$$

$$\theta_{t+1} = \theta_t - \alpha \cdot v^t_{\hat{v}}_t + \epsilon m^t_{\hat{m}}_t$$

Intuition:

- **Numerator** $m^t_{\hat{m}}_t$: direction (like momentum).
- **Denominator** $v^t_{\hat{v}}_t$

: scale (like RMSProp). If a coordinate has had **consistently large gradients**, its $v^t_{\hat{v}}_t$

- is big → smaller step for that coordinate.
- This makes the learning rate **adaptive per parameter**.

So in plain language:

Adam keeps a running “average gradient” and an “average squared gradient” for each weight. It uses the average gradient as a direction and divides by the average squared gradient’s square root to automatically shrink steps where gradients have been wild and enlarge where they’ve been calm. Then it corrects for early bias so these estimates behave nicely from the start.

3. Relationship to SGD, Momentum, RMSProp

You can see Adam as combining two earlier ideas:

SGD with Momentum

Momentum has:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

This is essentially just Adam's m_{t-1} part without adaptive scaling.

RMSProp

RMSProp keeps only the squared gradient average:

$$s_t = \beta s_{t-1} + (1 - \beta) g_t^2 \quad \theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{s_t} + \epsilon}$$

This is like Adam's denominator, but uses *raw* gradient g_t in the numerator instead of a momentum term.

Adam $\approx$ Momentum + RMSProp + bias correction

- Momentum-like behavior $\rightarrow$ from m_{t-1} .
 - RMSProp-like per-parameter scaling $\rightarrow$ from v_{t-1} .
 - Bias correction $\rightarrow$ prevents weird behavior at the start.
-

4. Typical hyperparameters and behavior

Common defaults (often literally used without changes):

- $\alpha = 0.001$
- $\beta_1 = 0.9$
- $\beta_2 = 0.999$
- $\epsilon = 10^{-8}$

Behavior:

- **Fast initial progress:** Adam usually “figures out” a good set of effective learning rates quickly.
- **Robust to rescaling of gradients:** If your loss or gradients are rescaled, Adam often handles it gracefully.

- **Less tuning:** Often works decently out-of-the-box, which is a big reason it became so widely used.

Caveat (you already bumped into this):

Adam vs AdamW: classic Adam mixes weight decay with the adaptive scaling in a slightly problematic way; AdamW decouples weight decay into a separate step (usually better for generalization). But conceptually, *both* use the same Adam core update above; AdamW just changes how L2 regularization is applied.

5. How to turn this into an *AI-enhanced learning tool* prompt

For your assignment, you need a **prompt/artifact** that your classmates can use plus an **interaction trace**. Here's a reusable prompt template you can drop into ChatGPT (or other LLMs) for "learning Adam":

Prompt you can share on Ed:

Title: Guided AI Tutor for Understanding the Adam Optimizer

You are an expert ML tutor helping me deeply understand the Adam optimizer.

Please follow this structure:

1. **Conceptual overview (no equations yet):** Explain Adam as if I know gradient descent and SGD with momentum, but not Adam. Use analogies and mention what problems Adam is trying to solve.
2. **Full equations:** Write down the full Adam update rules, define every symbol, and explain the role of each hyperparameter ($\alpha, \beta_1, \beta_2, \epsilon$).
3. **Step-by-step walkthrough:** Walk through a tiny 1D example (e.g., optimizing a simple quadratic) for 2–3 steps, showing how m_t, v_t , and the parameter update change over time.
4. **Comparisons:** Briefly compare Adam to SGD, SGD with Momentum, and RMSProp. When is Adam better or worse?

5. **Common pitfalls:** Explain situations where Adam might perform poorly or where people misuse it (e.g., setting learning rate too high, misunderstanding weight decay vs AdamW).
6. **Interactive check:** Ask me 3–5 conceptual quiz questions (increasing difficulty) to check my understanding, then wait for my answers. After I answer, correct me, give feedback, and add follow-up questions if needed.

Throughout:

- Keep explanations compact but precise.
- If something is uncertain or depends on research results (e.g., “Does Adam always generalize better than SGD?”), explicitly say that it’s nuanced and briefly summarize trade-offs rather than guessing.
- Do NOT fabricate experimental results, benchmarks, or citations. If you don’t know, say so.

Start with step 1.

You can then **run this prompt yourself**, copy the conversation, and:

- Highlight where the model:
 - **Does well:** clear explanation of bias correction, good analogies, correct equations.
 - **Potentially misleads / hallucinates:** e.g., if it overclaims about Adam “always” being better, or invents benchmark numbers without sources.
- Add your own annotations like:
 - “ Good: correct explanation of why bias correction is needed.”
 - “ Misleading: claims Adam always converges faster than SGD; true in many cases but not guaranteed; also some papers show SGD + momentum can generalize better for vision tasks.”

That gives you the “interaction trace + critical annotations” your assignment wants.

6. Quick concept check (for you right now)

Answer these in your own words (just to test yourself):

1. What are $\hat{m}_t$ and $\hat{v}_t$ in Adam, conceptually?
2. Why does Adam divide by $\sqrt{\hat{v}_t} + \epsilon$?
 1. $+\epsilon$?
2. What is the point of bias correction (the hats on $\hat{m}_t$ and $\hat{v}_t$)?
3. In what ways is Adam similar to both momentum and RMSProp?

If you'd like, you can answer these and I'll act as the "AI tutor," giving you feedback—this can double as the start of your trace.

You said:

Yes, I'm saying I want you to act as the AI tutor, okay can you do a couple things for me here: 1. can you point me to papers or videos that could help me learn about how Adam works or see its results in action

ChatGPT said:

Totally, I can be your Adam-tutor 😊

Let's start with what you asked for: **good papers + videos**, *with* a bit of guidance on how to use each for your assignment.

A. Core papers (to really understand Adam)

1. The original Adam paper (must-read)

Adam: A Method for Stochastic Optimization – Kingma & Ba (ICLR 2015) [arXiv+1](#)

- This is *the* Adam paper.
- What it's good for:
 - Formal definitions of the update rules and hyperparameters.

- Regret bound and convergence discussion.
 - Empirical results on benchmarks (MNIST, CIFAR-10 etc.) so you can see Adam vs SGD/RMSProp in action.
 - How to use it for your assignment:
 - Grab one or two plots (e.g., training loss vs iterations for Adam vs others) as examples of “results in action”.
 - Use the exact equations and wording as ground truth to catch LLM hallucinations about the algorithm.
-

2. Convergence + limitations of Adam

On the Convergence of Adam and Beyond – Reddi et al. (ICLR 2018, “AMSGrad” paper) [arXiv+2OpenReview+2](#)

- Shows that vanilla Adam can fail to converge even on simple convex problems.
- Proposes AMSGrad as a fix and analyzes convergence.
- Why it’s useful to you:
 - Great for “**critical viewpoint**” in your annotated trace: you can flag if the AI overclaims that “Adam always converges” or “Adam is always better than SGD”.
 - Gives you language about *when* Adam can misbehave.

You probably don’t need to grok every theorem proof, but knowing the **high-level failure mode** (effective learning rates not decaying properly) is nice.

3. Friendly overview that includes Adam among other optimizers

Sebastian Ruder – “An Overview of Gradient Descent Optimization Algorithms” (blog, but heavily cited) [Massachusetts Institute of Technology+1](#)

- Explains SGD, Momentum, Nesterov, Adagrad, RMSProp, Adam, etc., with pictures and pseudocode.
 - Great to build intuition and to compare Adam vs others.
 - Good for:
 - Sanity-checking how an LLM explains Adam's connection to RMSProp + Momentum.
 - Copying the comparison structure (pros/cons section) into your own understanding.
-

B. Videos to see Adam in action

Here are a few that are (1) clear, (2) math-light enough to get intuition, and/or (3) include code demos.

1. Conceptual + math explanation

“Adam Optimizer Explained in Detail | Deep Learning” (YouTube) [youtube.com+1](https://www.youtube.com/watch?v=UuUo0uE0v60)

- Walks through:
 - The idea of first and second moments.
 - Why bias correction is needed.
 - Equation breakdown with intuition.
 - Use it to:
 - Get a second explanation style after reading the paper.
 - Cross-check how they describe m_{t-1} , v_{t-1} , and bias correction versus what the LLM tells you.
-

2. Implementation from scratch

“Adam Optimizer from Scratch in Python” (multiple versions exist; e.g. short YT/online course tutorials) [youtube.com+2Class Central+2](https://www.youtube.com/watch?v=2ClassCentral+2)

- Typically:
 - Start with SGD.
 - Add momentum.
 - Add RMSProp.
 - Then combine into Adam and show code.
 - Why it’s nice for your assignment:
 - You can literally watch the parameter updates and loss curves.
 - If an LLM writes pseudocode for Adam, you can compare it against this code to catch subtle mistakes (missing bias correction, wrong order of operations, etc.).
-

3. “All optimizers in one place” style video

“Deep Learning – All Optimizers in One Video: SGD, Momentum, RMSProp, Adam...”
[youtube.com+1](https://www.youtube.com/watch?v=1)

- Great for:
 - Building intuition about *why* Adam is popular.
 - Understanding scenarios where plain SGD or SGD+Momentum might still be preferred (e.g., some vision tasks/generalization behavior).
-

C. How to plug these into your assignment

Since you need an **AI-enhanced learning tool + critical annotations**, here’s how I’d combine these resources with ChatGPT:

1. Use the Adam paper as ground truth.

- Keep the PDF open. When ChatGPT explains an equation, you quickly check whether:
 - The terms match Kingma & Ba's notation and order. [arXiv](#)
 - It correctly includes bias correction and ϵ .
- Any mismatch → note it in your trace as a possible hallucination or simplification.

2. Use Ruder's blog & the ICLR convergence paper to critique claims.

- If the AI says things like "Adam always converges faster than SGD" or "Adam is always the best choice," you can annotate:
 -  *Overclaim*: convergence is more nuanced; see Reddi et al. 2018 for counterexamples and AMSGrad fix. [arXiv+1](#)

3. Pair a YouTube implementation video with an LLM-generated code explanation.

- Ask ChatGPT to:
 - Implement Adam from scratch in NumPy.
 - Explain each line in relation to the math.
- Compare that code to one of the "from scratch in Python" videos.
- Annotate:
 -  *Good*: matches the bias-corrected update.
 -  *Potential issue*: LLM forgot the bias correction; video includes it. That's a concrete, easy-to-see hallucination/omission.

4. Use a video purely for intuition.

- Let ChatGPT explain Adam.
- Then watch a conceptual video.

- Go back to ChatGPT and *quiz it*: “You said X, but the video emphasized Y; can you reconcile that?”
- This back-and-forth is perfect for the “interaction trace” component.

If you’d like, next step I can:

- Design a **small hands-on experiment** (e.g., train a tiny neural net on a toy dataset with SGD vs Adam) and
- Give you a **prompt template + instructions** for how to run that with an AI tool and how to annotate hallucinations.

That would give you a very assignment-friendly artifact: “here’s a guided mini-lab + my AI conversation + my critiques.”

You said:

Can you give me some practice exam-style problems on Adam with solutions so that I can check my understanding

ChatGPT said:

Nice, let’s turn this into an Adam mini-quiz set. I’ll first list the **problems only**, then the **solutions** so you can try them like an exam.



Problems (try these first!)

Problem 1 – Interpreting Adam’s state

Adam maintains two vectors m_t and v_t and updates parameters as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

where $g_t = \nabla_{\theta} L(\theta_t)$

1. (a) In words, what does m_t approximate?
 (b) In words, what does v_t approximate?
 (c) Why do we divide by $1 - \beta_1^{t+1}$ and $1 - \beta_2^{t+1}$? What issue is that fixing?
-

Problem 2 – Adam vs SGD / Momentum / RMSProp

You're training a deep network on a messy real-world dataset. Compare the following optimizers:

- Plain SGD (single global learning rate)
- SGD with Momentum
- RMSProp
- Adam

Answer:

1. (a) Give one scenario where Adam is often preferred over plain SGD.
 (b) Give one reason someone might still choose SGD + Momentum over Adam, even if Adam seems to converge faster.
 (c) Very briefly state how Adam combines the ideas behind Momentum and RMSProp.
-

Problem 3 – One-step Adam numeric update (1D)

Consider a 1D parameter θ for the loss $L(\theta) = \frac{1}{2}\theta^2$. So the gradient is $g(\theta) = \theta$.

We use Adam with:

- $\alpha = 0.1$
- $\beta_1 = 0.9$
- $\beta_2 = 0.99$

- $\epsilon = 10^{-8}$
- initial $m_0 = 0, v_0 = 0, \theta_0 = 2.0$

At step $t=1$:

1. Compute the gradient g_1 .
2. Compute m_1 and v_1 .
3. Compute $\hat{m}_1$ and $\hat{v}_1$.
4. Compute θ_1 .
Give all values numerically (you can approximate square roots to 2–3 decimal places if needed).

Problem 4 – Spot the bug in Adam pseudocode

A friend writes the following “Adam” update for a single parameter:

```
m_t = beta1 * m_{t-1} + (1 - beta1) * g_t
v_t = beta2 * v_{t-1} + (1 - beta2) * g_t^2

theta_{t+1} = theta_t - alpha * (m_t / (sqrt(v_t) + eps))
```

They initialize $m_0 = 0, v_0 = 0$, and use this from step 1 onward.

1. (a) What important part of the Adam algorithm is missing?
(b) How could this omission affect early training behavior?

Problem 5 – Effects of hyperparameters

Assume you are using Adam with default values $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$.

1. (a) What is the qualitative effect of increasing β_1 from 0.9 to 0.99?
(b) What is the qualitative effect of decreasing β_2 from 0.999 to 0.9?

(c) What happens if you multiply the learning rate α by 10 (e.g., from 10^{-3} to 10^{-2}) while keeping everything else the same?

Problem 6 – Adam vs AdamW and weight decay

Classic Adam often applies L2 regularization by adding $\lambda \|\theta\|_2$ to the loss. AdamW applies “decoupled weight decay” as:

$$\theta_{t+1} = \theta_t - \alpha \hat{m}_t + \epsilon - \alpha \lambda \theta_t$$

$$\hat{m}_{t+1} = \beta_1 \hat{m}_t + (1 - \beta_1) \hat{g}_t$$

$$\hat{v}_{t+1} = \beta_2 \hat{v}_t + (1 - \beta_2) \hat{g}_t^2$$

1. (a) Conceptually, what’s the difference between **L2 regularization inside the loss** vs **decoupled weight decay** in AdamW?
 (b) Why can decoupled weight decay (AdamW) behave better than standard Adam + L2 in practice?
-

Solutions

Solution 1 – Interpreting Adam’s state

1. (a) $\hat{m}_t$ is an **exponential moving average of the gradient**.
 - It approximates the *first moment* (mean) of the gradient.
 - Intuitively: a smoothed, momentum-like gradient direction.
2. (b) $\hat{v}_t$ is an **exponential moving average of the squared gradient**.
 - It approximates the *second raw moment* (mean of squared gradients).
 - Intuitively: it tracks how large the gradients have been for each parameter, so it can scale learning rates down where gradients are consistently large.
3. (c) We divide by $1 - \beta_1^{t+1}$ and $1 - \beta_2^{t+1}$ to get **bias-corrected** estimates $\hat{m}_t$ and $\hat{v}_t$.

- Because we start with $m_0 = 0$ and $v_0 = 0$, early values are biased toward zero (we haven't had time to "fill in" the moving average).
 - The factors $1 - \beta_1$ and $1 - \beta_2$ correct this, so $\hat{m}_t$ and $\hat{v}_t$ are closer to the true expected mean and mean-square of the gradient during the early steps.
-

Solution 2 – Adam vs SGD / Momentum / RMSProp

1. (a) Adam is often preferred over plain SGD when:

- The problem is **high-dimensional and messy**, like deep nets with different layers having very different gradient scales.
- You want **less learning rate tuning**, because Adam adapts learning rates per parameter.
- Data is **noisy** (e.g., mini-batch SGD in deep learning); Adam's adaptive steps can be more stable.

2. Any one of those is fine.

(b) Reasons to still choose SGD + Momentum:

- **Better generalization** in many practical settings (e.g., image classification) where SGD + momentum is still state-of-the-art.
- Simpler behavior and **fewer moving parts**, easier to reason about.
- Some empirical studies show Adam can get stuck in sharp minima or have poorer final test accuracy, even if it optimizes training loss faster.

3. (c) How Adam combines ideas:

- From **Momentum**: a smoothed average of gradients (m_t), which gives a stable direction.
- From **RMSProp**: a smoothed average of squared gradients (v_t), which produces adaptive per-parameter learning rates through the division by $\sqrt{v_t}$.

- .
 $\Rightarrow \text{Adam} \approx \text{Momentum} + \text{RMSProp} + \text{bias correction}.$

Solution 3 – One-step Adam numeric update (1D)

We have $L(\theta) = \frac{1}{2}\theta^2$, so $g(\theta) = \nabla_{\theta} L(\theta) = \theta$.

Given:

- $\theta_0 = 2.0$
- $m_0 = 0, v_0 = 0$
- $\alpha = 0.1, \beta_1 = 0.9, \beta_2 = 0.99, \epsilon = 10^{-8}$

At step $t=1$:

1. Gradient:

$$g_1 = \nabla_{\theta} L(\theta_0) = \theta_0 = 2.0$$

2. Update the moving averages:

$$\begin{aligned} m_1 &= \beta_1 m_0 + (1 - \beta_1) g_1 = 0.9 \cdot 0 + 0.1 \cdot 2 = 0.2 \\ v_1 &= \beta_2 v_0 + (1 - \beta_2) g_1^2 = 0.99 \cdot 0 + 0.01 \cdot (2^2) = 0.04 \end{aligned}$$

3. Bias-corrected estimates:

$$\begin{aligned} \hat{m}_1 &= \frac{m_1}{1 - \beta_1} = \frac{0.2}{1 - 0.9} = 2.0 \\ \hat{v}_1 &= \frac{v_1}{1 - \beta_2} = \frac{0.04}{1 - 0.99} = 4.0 \end{aligned}$$

4. Parameter update:

$$\theta_1 = \theta_0 - \alpha m^1 v^1 + \epsilon \quad \theta_1 = \theta_0 - \alpha \frac{\hat{m}_1}{\sqrt{\hat{v}_1}} + \epsilon$$

$$\text{We have } v^1 = 4.0 = 2.0 \sqrt{\hat{v}_1} = \sqrt{4.0} = 2.0 v^1$$

$$= 4.0$$

$$= 2.0.$$

$$\theta_1 \approx 2.0 - 0.1 \cdot 2.0 \cdot 2.0 + 10^{-8} \approx 2.0 - 0.1 \cdot 1 = 1.9 \quad \theta_1 \approx 2.0 - 0.1 \cdot \frac{2.0}{2.0 + 10^{-8}} \approx 2.0 - 0.1 \cdot 1 = 1.9$$

(More precisely: $2.0 - 0.1 \times 0.999999995 \approx 1.9000000005$, $2.0 - 0.1 \times 0.999999995 \approx 1.9000000005$, but 1.9 is fine.)

So the answers:

- $g^1 = 2.0$, $g_1 = 2.0$, $g^1 = 2.0$
- $m^1 = 0.2$, $m_1 = 0.2$, $v^1 = 0.04$, $v_1 = 0.04$, $v^1 = 0.04$
- $\hat{m}^1 = 2.0$, $\hat{m}_1 = 2.0$, $\hat{m}^1 = 2.0$, $\hat{v}^1 = 4.0$, $\hat{v}_1 = 4.0$, $\hat{v}^1 = 4.0$
- $\theta_1 \approx 1.9$, $\theta_1 \approx 1.9$, $\theta_1 \approx 1.9$

Solution 4 – Spot the bug in pseudocode

1. (a) The missing part is **bias correction**:

There is no computation of $m^t = m_t / (1 - \beta_1^t)$ and $v^t = v_t / (1 - \beta_2^t)$. The update should be:

$m_t = \dots$

$v_t = \dots$

$\hat{m}_t = m_t / (1 - \beta_1^t)$

$\hat{v}_t = v_t / (1 - \beta_2^t)$

$\theta_{t+1} = \theta_t - \alpha * (\hat{m}_t / (\sqrt{\hat{v}_t}) + \epsilon)$

(b) Without bias correction, early in training $m_{t,m}$ and $v_{t,v}$ are **biased toward zero** (because they started at zero).

Consequences:

- The effective step sizes in the first few iterations can be **too small or inconsistent**, so Adam might:
 - warm up more slowly,
 - behave differently from the intended theory/implementation,
 - be more sensitive to initialization and early gradients.

In practice, modern frameworks always include this bias correction.

Solution 5 – Effects of hyperparameters

1. (a) Increasing β_1 from 0.9 $\rightarrow$ 0.99:

- $m_{t,m}$ becomes **more heavily smoothed**: it remembers the past gradients for longer.
- Pros: more stable direction, can reduce noise.
- Cons: reacts more slowly to changes; if the gradient direction shifts, Adam will lag behind.

2. (b) Decreasing β_2 from 0.999 $\rightarrow$ 0.9:

- $v_{t,v}$ reacts **more quickly** to changes in the squared gradient (less smoothing).
- That means the adaptive learning rates can change rapidly:
 - Pros: more responsive to sudden changes in gradient magnitude.
 - Cons: more noisy, may cause unstable or jittery updates.

3. (c) Multiplying α by 10:

- Increases the **overall step size** of Adam.

- Even though Adam has adaptive scaling, too large an α can still:
 - cause divergence,
 - make loss oscillate or explode.
- Sometimes you can get faster convergence with a slightly larger learning rate, but 10x is often risky unless you're sure the problem tolerates it.

Solution 6 – Adam vs AdamW and weight decay

1. (a) Conceptual difference:

- **L2 regularization in the loss:**
 You add $\frac{\lambda}{2} \|\theta\|^2$ to the loss, so the gradient becomes:

$$g_{\text{total}} = g_{\text{data}} + 2\lambda\theta$$

$$g_{\text{total}} = g_{\text{data}} + 2\lambda\theta$$
 That L2 term is then passed into Adam's adaptive machinery (the same $m_t, v_t, m_{\hat{t}}, v_{\hat{t}}$, scaling, etc.).
- **Decoupled weight decay (AdamW):**
 You **separate** the weight decay from the adaptive gradient step:

$$\theta_{t+1} = \theta_t - \alpha \hat{v}_t + \epsilon - \alpha \lambda \theta_t$$

$$\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \alpha \lambda \theta_t$$

$$\theta_{t+1} = \theta_t - \alpha \hat{v}_t + \epsilon - \alpha \lambda \theta_t$$
 So the regular gradient uses adaptation, but the decay term $-\alpha \lambda \theta_t$ is a **direct shrink** of weights, independent of the adaptive scaling.

2. (b) Why AdamW can be better:

- In standard Adam + L2, the regularization effect gets **entangled with the adaptive learning rates**: parameters with large or small v_t get different effective decay strengths, which is not what we usually want.
- In AdamW, weight decay is applied **uniformly** in parameter space (relative to the learning rate) and is not distorted by the adaptive scaling in v_t .
- This more closely matches the behavior of “true” weight decay used in SGD with momentum and often leads to **better generalization and more predictable**

tuning.

If you want, next we can:

- Turn some of these into an **interactive quiz** (I ask, you answer, I correct),
- Or design **one big “exam problem”**: e.g., comparing trajectories of Adam vs SGD on a toy function, with you interpreting the behavior.

- cause divergence,
- make loss oscillate or explode.
- Sometimes you can get faster convergence with a slightly larger learning rate, but 10x is often risky unless you're sure the problem tolerates it.

Solution 6 – Adam vs AdamW and weight decay

1. (a) Conceptual difference:

- **L2 regularization in the loss:**

You add $\lambda \|\theta\|^2$ to the loss, so the gradient becomes:

$$g_t^{\text{total}} = g_t^{\text{data}} + 2\lambda\theta_t$$

That L2 term is then passed into Adam's adaptive machinery (the same m_t , v_t , scaling, etc.).

- **Decoupled weight decay (AdamW):**

You **separate** the weight decay from the adaptive gradient step:

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \alpha\lambda\theta_t$$

So the regular gradient uses adaptation, but the decay term $-\alpha\lambda\theta_t$ is a **direct shrink** of weights, independent of the adaptive scaling.

(b) Why AdamW can be better:

- In standard Adam + L2, the regularization effect gets **entangled with the adaptive learning rates**: parameters with large or small v_t get different effective decay strengths, which is not what we usually want.
- In AdamW, weight decay is applied **uniformly** in parameter space (relative to the learning rate) and is not distorted by the adaptive scaling in v_t .
- This more closely matches the behavior of “true” weight decay used in SGD with momentum and often leads to **better generalization and more predictable tuning**.

If you want, next we can:

- Turn some of these into an **interactive quiz** (I ask, you answer, I correct),
- Or design **one big “exam problem”**: e.g., comparing trajectories of Adam vs SGD on a toy function, with you interpreting the behavior.

📄 👍 🗨️ ↶ ↷ ...

Is this conversation helpful so far? 👍 🗨️ ✕

Ask anything

+ 🔄 Thinking ▾



ChatGPT can make mistakes. Check important info.